

# forget-lah | Team quick start

**Set up your workspace. Run the app. Test the follow-up cases.**

Windows guide | Foundation v0.1.0 | 10 September 2026

## Start here

Follow this guide to run forget-lah on your own computer. It covers the working **local foundation (M1)**: staff login, three fictional patients, follow-up detection and processing evidence. Live agent conversations and patient channels are the next implementation milestone.

## Before you begin

1. Have access to the [forget-lah repository](#) using your own GitHub account. You should be able to see the source files, including `compose.yaml` and `scripts`.
2. Use a Windows computer with an internet connection and permission to install VS Code, Git and Docker Desktop. Installation instructions are on page 2.
3. Allow time for downloads and a possible Windows restart.

## Follow the pages in order

| Page | Team instructions                                         |
|------|-----------------------------------------------------------|
| 2    | Install the tools and start Docker Desktop                |
| 3    | Download the code, generate local credentials and sign in |
| 4    | Test dental, myopia and antenatal follow-up cases         |
| 5    | Check refresh, login, sign-out and evidence labels        |
| 6    | Run automated tests, stop/restart and get code updates    |
| 7    | Troubleshoot a problem and report your test results       |

**Success looks like this:** three cases, with totals of 1 overdue recall, 1 upcoming visit and 1 missed appointment. All three show **Ready for agent**. This means the cases are prepared; no patient has been contacted.

## Your local workspace

The browser address is <http://localhost:8080>. `localhost` means your own computer. Each teammate runs a separate local copy with its own database and generated passwords.

Keep your generated `.env` file private. Use only your own `DEMO_STAFF_EMAIL` and `DEMO_STAFF_PASSWORD` values to sign in.

## Which document should I use?

Use this PDF for reading or `TEAM_START_HERE.md` for editable instructions. Press **Ctrl+Shift+V** in VS Code to preview Markdown. The `docs/README.md` index explains the supporting documents.

# 1. Prepare your computer

This walkthrough uses **Windows and PowerShell**. Start with an internet connection and permission to install software. Downloads and a restart can take longer than the app setup itself.

## Step 1. Install the three tools

Use the official installation pages below. Choose the installer that matches your computer, follow its prompts, and reopen VS Code after installation.

| Tool           | What it is for                                                                                            | Official page                              |
|----------------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------|
| VS Code        | Open the project and enter the commands in its terminal.                                                  | <a href="#">VS Code for Windows</a>        |
| Git            | Download the team's code and keep track of changes.                                                       | <a href="#">Git for Windows</a>            |
| Docker Desktop | Run the app and database in packaged environments called containers. Use its WSL 2/Linux container setup. | <a href="#">Docker Desktop for Windows</a> |

For this Docker route, you do **not** need to install Python, Node.js or PostgreSQL separately. No AWS account, organiser API key or Codex/ChatGPT subscription is required to run M1.

## Step 2. Check WSL if Docker reports a problem

**WSL** lets Windows run the Linux environment used by Docker. If Docker is already working, skip the installation command below.

Open PowerShell and check:

```
wsl --version
```

If WSL is missing, open **Start > PowerShell > Run as administrator**, then run:

```
wsl --install --no-distribution
```

Save your work and **restart Windows** when installation finishes or Windows requests it. Then open Docker Desktop. This installs the WSL runtime without adding a separate Ubuntu installation for this guide. If Docker reports an outdated WSL version, follow its update instructions; `wsl --update` is the standard update command.

Sources: [Microsoft WSL installation](#) and [WSL command reference](#).

## Step 3. Start Docker Desktop

Open **Docker Desktop** from Start, complete its first-run prompts and wait for the engine to run. Use **Linux containers**. If it reports disabled virtualization, ask your team's platform owner or IT support to enable the required machine setting.

**Checkpoint:** Docker Desktop opens and its engine is running. Leave it open while using the app. Normal project commands on the next page run in a regular VS Code PowerShell terminal.

## 2. Download and launch for the first time

**Before starting:** confirm you have repository access. If you already have the code, skip Step 1 and open the existing project folder.

### Step 1. Download the code once

In File Explorer, create C:\working\projects if it does not exist. In VS Code, open that parent folder and choose **Terminal > New Terminal**. Select PowerShell. Run:

```
Set-Location 'C:\working\projects'  
git clone https://github.com/satworkz/forget-lah.git
```

Sign in to GitHub in the browser if prompted. Git creates a new forget-lah folder. Do not create a second nested copy. [GitHub's cloning guide](#) explains downloading a repository and access requirements.

### Step 2. Open the correct folder

Choose **File > Open Folder > C:\working\projects\forget-lah**. Open a new terminal. You should see compose.yaml, scripts, apps and src in the left file list. The terminal path should end in \forget-lah>.

Copy only the commands below, not the PS C:\...> prompt. Run one line at a time. If one fails, resolve it before continuing.

### Step 3. Check, configure, then start

```
./scripts/dev.ps1 doctor  
./scripts/dev.ps1 setup  
./scripts/dev.ps1 up  
./scripts/dev.ps1 status
```

| Command | What happens / success sign                                                                                        |
|---------|--------------------------------------------------------------------------------------------------------------------|
| doctor  | Checks Git, Docker and Linux containers. Ends with <b>Prerequisites passed</b> .                                   |
| setup   | Creates .env with random local passwords. If it already exists, preserves it.                                      |
| up      | Downloads/builds the app, creates the database tables and starts the services. First run can take several minutes. |
| status  | Shows running services. bootstrap finishing with <b>Exited (0)</b> is normal. See page 6.                          |

### Step 4. Sign in

Open <http://localhost:8080>. In VS Code, open .env. Copy the value **after the equals sign** for DEMO\_STAFF\_EMAIL and DEMO\_STAFF\_PASSWORD into the login fields. The email is normally staff@forget-lah.example; the generated password is different on each laptop.

Click **Open workspace**. After about ten seconds, click **Refresh cases**. Expect three fictional patients. Keep .env private. Editing passwords in that file does not reset credentials already stored in the database.

### 3. Test the three patient follow-up cases

Sign in and open **Patient follow-up**. This test checks that the same foundation recognises three different reasons to follow up. You do not need to type patient data or connect an external clinic system.

#### Test A. Dental: a routine check-up is overdue

1. Find **Mr Lim (demo)** in the table.
2. Check **Specialty: Dental, Follow-up reason: Overdue recall**, and **Stage: Ready for agent**.
3. Click **View evidence** on Mr Lim's row. Scroll down if necessary.
4. Confirm the evidence panel names Mr Lim and shows **CASE IDENTIFIED**, then **FOUNDATION CASE READY**, both labelled **Application rule**.
5. Check the note says **Ready for the agent runtime milestone; no outreach sent**. Click **Close**.

**Why this case exists:** the mock record says a routine recall was due 14 days ago and there is no future booking. This is a check-up that was not arranged, rather than a recorded no-show.

#### Test B. Myopia: an appointment is coming up

1. Find **Alex (demo)**.
2. Check **Specialty: Myopia, Follow-up reason: Upcoming visit**, and **Stage: Ready for agent**.
3. Click **View evidence** on Alex's row. Confirm Alex is named, with the same two application-rule events and the no-outreach note. Close the panel.

**Why this case exists:** the mock source supplies a scheduled appointment three days ahead. M1 identifies it for follow-up. It does not yet send a reminder or give visit-preparation instructions.

#### Test C. Pregnancy: a visit was missed

1. Find **Priya (demo)**.
2. Check **Specialty: Antenatal, Follow-up reason: Missed appointment**, and **Stage: Ready for agent**. Antenatal means care during pregnancy.
3. Click **View evidence** on Priya's row. Confirm Priya is named, with the same two application-rule events and the no-outreach note. Close the panel.

**Why this case exists:** the mock source explicitly marks yesterday's appointment **no-show**. The app does not assume a patient missed a visit just because its time has passed. No clinical urgency decision is made in this test.

#### Expected result after all three tests

| Overdue recall | Upcoming visit | Missed appointment | Total cases |
|----------------|----------------|--------------------|-------------|
| 1              | 1              | 1                  | 3           |

**Ready for agent means the case has been prepared for the next milestone.** It does not mean a patient was contacted, a booking was confirmed, or follow-up was completed. Fixture dates are relative to the day they are supplied; you do not need to change dates manually to start the demo.

## 4. Test the rest of the staff experience

These checks need only the browser. Record **Pass / Fail / Not tried** for each item. Use the exact address `http://localhost:8080` throughout.

### Test D. Refresh without creating duplicates

1. After the three cases appear, click **Refresh cases** three times, allowing each refresh to finish.
2. Wait about ten seconds and refresh again.
3. Expect **three cases**, not six or nine, with one case for each patient.

The button reloads the list. Source checks run in the background; re-reading the same source episode should not create another case.

### Test E. Reload without losing your login

1. While signed in, press **F5** or click the browser reload button.
2. Expect the staff workspace to return without entering your password again, provided the session is still valid.

### Test F. Sign out and protect the workspace

1. Click **Sign out** in the left sidebar.
2. Expect the **Clinic workspace** login screen.
3. Reload the page. It should remain on the login screen.
4. Sign in again with your local credentials. Expect the same three cases.

### Test G. Reject an incorrect password

1. Sign out. Use the correct demo email and a deliberately incorrect password **once**.
2. Click **Open workspace**. Expect **Invalid credentials**; the cases should not open.
3. Replace it with the correct password and sign in successfully.

Repeated attempts are rate-limited. If you hit the limit, wait at least a minute before retrying; avoid repeatedly clicking the login button.

### Test H. Check the implementation boundary

Scroll to **The agent team**. Expect all three cards to say **Planned**. No chat box, patient message, live reasoning trace or completed booking should appear in this release. Application-rule evidence must not be presented as Claude's reasoning.

### Short team walkthrough script

"We have a running staff workspace with three fictional cases. Dental covers an overdue check-up, myopia covers an upcoming visit, and antenatal covers an explicit missed appointment. Each case has visible source and processing evidence. Today the system prepares cases using application rules. Our next milestone connects the agent team to decide and carry out permitted follow-up actions."

For the hackathon, this is a **foundation walkthrough**, not yet the final agent demonstration.

## 5. Automated checks and everyday use

All commands on this page run in the project folder's PowerShell terminal with Docker Desktop running.

### Run the automated checks

```
./scripts/dev.ps1 test
```

This builds and runs the test container. The verified version has **20 tests**, including a real PostgreSQL check that two workers cannot claim the same job. Other checks cover login, clinic separation, duplicate prevention, source rules, job recovery and strict decision validation.

**Pass:** tests reach 100%, no failures are reported, and the command finishes successfully. Quiet output may show dots rather than a full "20 passed" sentence. **Fail:** FAILED, ERROR, or a non-zero Docker exit message. Some dependency deprecation warnings can appear even when tests pass.

Tests use isolated temporary databases/schemas, including a separate PostgreSQL test database. They do not send patient messages. These 20 engineering tests are separate from the planned 60 agent evaluation scenarios; they do not validate live Claude or provider integrations.

### Understand service status

```
./scripts/dev.ps1 status
```

| Service              | Expected status after startup                                     |
|----------------------|-------------------------------------------------------------------|
| db, api, mock-clinic | Running / Up, with <b>healthy</b> shown                           |
| worker, web          | Running / Up; these do not show a health-check label              |
| bootstrap            | <b>Exited (0):</b> its database setup task completed successfully |

### Stop at the end of the day

```
./scripts/dev.ps1 down
```

This stops the app while keeping its local database. Closing only VS Code does not stop the containers. Do not delete the Docker database volume or use a factory reset as a normal shutdown step.

### Start again or verify restart behaviour

Open Docker Desktop, then run `./scripts/dev.ps1 up`. Open the browser and sign in if asked. Refresh cases and expect the same three cases. This is a useful additional manual persistence check for teammates to record.

### Get a teammate's latest changes

First run `git status`. If it reports your own changes, ask the code owner to help save them. If your working copy is clean and the first shared commit exists, run `git pull --ff-only`, then `./scripts/dev.ps1 up` to rebuild. Run the automated checks again after code changes. Stop and ask for help if Git reports a conflict or divergence; do not discard local work.

## 6. If something goes wrong

Start with the matching row. Keep the error message; it usually tells the team where to look.

| What you see                                          | What to do                                                                                                                                             |
|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Docker engine unavailable, named-pipe or server error | Open Docker Desktop. For WSL errors, follow page 2 and complete any pending restart. Then retry doctor.                                                |
| Empty repository or no scripts folder                 | Confirm the repository contains the source and that you have access. Open the folder containing <code>compose.yaml</code> .                            |
| <code>dev.ps1</code> not found                        | Use VS Code <b>File &gt; Open Folder</b> to open <code>forget-lah</code> , then open a new PowerShell terminal.                                        |
| PowerShell says scripts are disabled                  | Send the policy error to your platform owner or IT support. Do not disable security settings globally.                                                 |
| First build fails while downloading                   | Check the internet connection and failed download. Retry up when the connection is restored.                                                           |
| Browser cannot open the app                           | Check status and use <code>http://localhost:8080</code> . If port 8080 is in use, ask the platform owner to resolve the conflict.                      |
| Origin not allowed                                    | Use <code>localhost:8080</code> , not <code>127.0.0.1:8080</code> or a different port. The configured address is checked by the API.                   |
| Invalid credentials                                   | Use this laptop's <code>.env</code> values after the equals signs, without extra spaces. Editing <code>.env</code> does not reset an existing account. |
| Login attempts limited                                | Wait at least one minute, then try once with the correct credentials.                                                                                  |
| Zero cases, or only one evidence event                | Wait about ten seconds, refresh cases, then reopen evidence. If it persists, inspect status and logs below.                                            |
| Database error after editing <code>.env</code>        | Contact the platform owner. The saved database keeps its original credentials; do not delete its volume.                                               |
| No messages or agent replies                          | Expected in M1. The next milestone implements those capabilities.                                                                                      |

### Collect useful diagnostics

```
./scripts/dev.ps1 status
./scripts/dev.ps1 logs
```

`logs` shows recent API, worker and database-setup messages. Share the relevant error after checking it contains no private values. Never attach `.env`, credentials or real patient information.

### Send this short test report to the team

**Tester / date / version or commit:** ...

**Results:** Startup ...; Cases A-C ...; Checks D-H ...; Automated tests ...; Restart ... . Use **Pass / Fail / Not tried** for each.

**If something failed:** steps taken; expected result; actual result; relevant error or screenshot with private details removed.